

Senior React Developer — Szakmai kérdés-válasz

A legvalószínűbben előkerülő technikai kérdések tudástára, olyan válaszokkal, amelyek a rákérdezéseket is kibírják. A legvalószínűbbtól a nichebb témák felé haladva.

Fókuszban lévő stack: React ökoszisztéma és belső működés · Modern JavaScript/TypeScript · Jest + React Testing Library · Skálázható frontend architektúra és megosztott komponenskönyvtárak · SPFx / SharePoint Online integráció.

TARTALOM

1. React belső működés és renderelés — reconciliation, keyek, StrictMode, fiber
2. Hookok mélyen — deps, closure-ök, useMemo/useCallback, saját hookok
3. Teljesítmény és optimalizálás — memo, re-renderek, code splitting, profilozás
4. State-kezelés és adatlekérés — context, server state, cache
5. Modern JavaScript és TypeScript — async, closure, generikusok, immutabilitás
6. Tesztelés — Jest és React Testing Library — queryk, async, mockolás
7. Skálázható architektúra és komponenskönyvtárak — design systemek, határok
8. SPFx és SharePoint Online — web partok, lifecycle, PnPjs, jogosultságok
9. Akadálymentesség és frontend minőség — a11y, biztonsági érintkezési pontok

1 React belső működés és renderelés

Az interjúztató azt akarja látni, hogy érted, **miért** viselkedik így a React — nem csak az API-t ismered.

K Mi történik valójában, amikor egy komponens újrenderelődik?

A „render” azt jelenti, hogy a React **meghívja a komponensfüggvényedet**, hogy előállítson egy új elemfát (a virtuális DOM-ot). Ezután a React **összeveti (reconcile)** ezt a fát a korábbival, kiszámolja a szükséges minimális DOM-módosításokat, és commitolja őket. A renderelés nem feltétlenül nyúl a DOM-hoz — egy olyan render, ami azonos kimenetet ad, semmit sem commitol.

A megfogalmazandó mentális modell: *render fázis* (tisztá, megszakítható/újraindítható, mellékhatás nélkül) vs *commit fázis* (a DOM módosul, refek beállnak, a layout effectek szinkron lefutnak, a passzív effectek ütemeződnek).

Rákérdezés-csapda: „Egy state-változás mindig újrenderelést okoz?” — Nem. Ha ugyanarra a primitív értékre állítod a state-et (`Object.is` egyenlőség), a React kihagyja az adott komponens újrenderelését.

K Magyarázd el a reconciliation-t és a diffing heurisztikákat.

A React diffje azért $O(n)$, mert két feltételezésre épít: (1) a **különböző típusú elemek különböző fát eredményeznek** — egy `<div>` → `<span>` váltás lebontja a részfat és újraépíti; (2) a **keyek** megmondják a Reactnek, hogy mely gyerekek stabilak a renderek között. Azonos típus + azonos pozíció → a React újrahasználja a DOM-node-ot, és csak a változott propokat frissíti.

K Miért fontosak a keyek, és miért probléma az index-mint-key?

A keyek stabil identitást adnak minden listaelemnek, hogy a React párosítani tudja a régi és új gyerekeket. **Index-keyekkel** egy beszúrás vagy átrendezés minden indexet eltol, így a React azt hiszi, az elemek helyben változtak — rossz DOM-node-okat használ újra. Tünetek: rossz sorhoz tartozó input-értékek, elvesztett fókus, elromlott animációk, beragadt uncontrolled state.

Használj stabil domain ID-t. Az index-key csak statikus, csak-hozzáfűzős, sosem átrendezett, lokális state nélküli listánál biztonságos.

K Mi a Fiber architektúra, és miért létezik?

A Fiber a React magjának újrainplementációja (v16 óta), ami **megszakíthatóvá** teszi a renderelést. Minden elemhez tartozik egy fiber node, ami a state-ét tárolja, és a munka egységekre bomlik, így a React szüneteltetni, priorizálni és folytatni tud — ez teszi lehetővé a concurrent funkciókat. A Fiber előtt a reconciliation szinkron, rekurzív stack volt, amit nem lehetett megszakítani, így egy nagy update leblokkolhatta a fő szálát.

K Miért hívja meg a StrictMode kétszer a komponenseimet fejlesztés közben?

Hogy felszínre hozza a **tisztátalan rendereket és a nem biztonságos effecteket**. A React szándékosan kétszer hívja meg a render-függvényeket és az effect setup/cleanup-ot (csak dev módban), így a renderbe rejtett mellékhatások vagy a hiányzó effect-cleanup láthatóvá válnak. Ha a kétszeri hívás elrontja a komponensedet, az valódi bug — jellemzően render közbeni mutáció vagy nem idempotens effect.

Ezt mondd: Csak dev módban fut, produkcióban nem, de előrevetíti a concurrent renderelés viselkedését, ahol a komponensek commit nélkül is renderelődhetnek.

K Controlled vs uncontrolled komponensek — mikor melyiket választanád?

Controlled: az érték a React state-ben él, egyetlen igazságforrás, akkor kell, ha validálsz gépelés közben, feltételesen formázol, vagy egymástól függő mezőid vannak. **Uncontrolled:** a DOM tartja az értéket, refen keresztül olvasod — könnyebb, jó egyszerű űrlapokhoz, fájl-inputhoz, vagy nem-React widgetek integrálásához. Enterprise űrlapoknál gyakran controlled irányba mész egy form library-vel (React Hook Form), ami a legtöbb inputot teljesítmény miatt uncontrolledként tartja, miközben validál.

2 Hookok mélyen

Számíts closure- és dependency-array-kérdésekre — itt látszik a seniorság.

K Mik a Hookok szabályai, és mi törik el, ha figyelmen kívül hagyod őket?

A hookokat **csak a legfelső szinten** hívd (soha feltételben, ciklusban vagy beágyazott függvényben), és **csak React függvényekből**. A React a hívási sorrend, nem a név alapján követi a hook-state-et — egy feltételes hook eltolja a sorrendet a renderek között, így a React rossz hookhoz köti a state-et, és minden összeomlik. Pontosan ezért létezik a linter szabály.

K Magyarázd el a stale closure problémát useEffect-nél.

Minden render friss függvénypéldányokat hoz létre, amelyek „lezárják” (close over) az adott render propjait/state-jét. Ha egy effect elkap egy értéket, de az nincs benne a dependency array-ben, az effect továbbra is a **régi elkapott értéket** látja — ez a klasszikus stale closure. Példa: egy `setInterval`, ami olvassa a `count`-ot, de `[]` deps-e van, örökre a kezdeti count-ot fogja logolni.

Megoldások: tedd be az értéket a deps-be, vagy használd a **functional updatert** `setCount(c => c + 1)`, hogy ne függj az elkapott értéktől, vagy tartsd egy refben.

Rákérdezés-csapda: „Miért ne hagyd egyszerűen ki a dependency array-t?” — Akkor az effect minden renderkor lefut, ami általában minden billentyűleütésre újra-feliratkozik/újra-lekér. A deps nem javaslat; a helyességet definiálja.

K useMemo vs useCallback — mi a valódi különbség, és mikor számítanak?

A `useMemo` egy **kiszámolt értéket** memoizál; a `useCallback` egy **függvényreferenciát**. A `useCallback(fn, deps)` szó szerint `useMemo(() => fn, deps)`.

Két esetben számítanak: (1) egy drága számítás, amit nem akarsz minden renderkor újra elvégezni; (2) egy érték/függvény, amit egy `React.memo` gyereknek adsz át, vagy egy másik hook dependency-jeként használsz, ahol a renderenkénti új referencia meghíúsítaná a memoizálást vagy újra elindítaná az effecteket. A triviális értékek becsomagolása nettó veszteség — a cache-nyilvántartás többbe kerül, mint amennyit megspórol.

Senior jelzés: Említsd meg, hogy a React Compiler (React 19+) automatikusan memoizál, csökkentve a manuális `useMemo` / `useCallback` szükségességét — de a mechanikát így is érteni kell.

K useEffect vs useLayoutEffect?

A `useLayoutEffect` **szinkron fut a DOM-módosítások után, de a böngésző kirajzolása (paint) előtt** — akkor használd, ha layoutot kell olvasnod (megmérni egy node-ot) és módosítanod, mielőtt a felhasználó villódzást látna. A `useEffect` **a paint után** fut, aszinkron módon — ez az alapértelmezett adatlekéréshez, feliratkozásokhoz, logoláshoz. A layout effectek túlhasználata blokkolja a paintet és rontja a teljesítményt.

K Mikor nyúlsz a useRef-hez?

Két felhasználás: (1) egy DOM-node imperatív elérése (fókusz, görgetés, mérés, third-party lib integrálása); (2) egy **mutálható érték tartása, ami rendreken átível anélkül, hogy újrenderelést váltana ki** — timerek, korábbi értékek, „legfrissebb” callbackek. Lényeg: a `ref.current` mutálása nem okoz renderelést, pontosan ezért nem való semmihez, amit a UI megjelenít.

K Hogyan tervezel jó saját hookot?

Egy saját hook csak egy `use`-zal kezdődő függvény, ami más hookokat kombinál. A jók **egyértelmű, egyetlen felelősséggel** bírnak (pl. `useDebounce`, `useLocalStorage`, `usePagination`), stabil, jól tipizált interfészt adnak vissza, és úgy kapszulázzák be az effect/cleanup-ot, hogy a hívók ne tudják félrehasználni. *Logikát* emelnek ki, nem markupot — ez a különbség a komponenshez képest.

3 Teljesítmény és optimalizálás

Az enterprise React appok belassulnak; a szisztematikus megközelítésre kíváncsiak, nem random mikro-trükkökre.

K Egy oldal túl sokat renderelődik. Hogyan diagnosztizálod?

Mondd el a módszert, sorrendben:

- **Először mérd** — React DevTools Profiler, hogy lásd, mely komponensek renderelődnek és miért („mi okozta ezt a rendert”). Ne megérzésre optimalizálj.
- Azonosítsd a kiváltó okot: szülő state-változás lecsorog lefelé, egy új objektum/tömb/függvény prop meghíúsítja a memoizálást, vagy a context value változik.
- Alkalmazd a megfelelő megoldást: `React.memo` tiszta gyerekekre, propok stabilizálása `useMemo` / `useCallback`-kel, state kiemelése vagy colocatio-n-je, context szétbontása.
- Mérd újra a megerősítéshez.

K Mit csinál a `React.memo`, és mik a korlátai?

Memoizál egy komponenst, így kihagyja az újrenderelést, ha a propok **shallow-egyenlők** az előző renderhez képest. Korlátok: csak akkor segít, ha a propok valóban stabilak — adj át egy friss inline objektumot/tömböt/callbacket, és így is újrenderelődik. Azt sem állítja meg, ha a renderelést a komponensen *belüli* `useState` / `useContext` okozza. Célzott eszköz, nem általános becsomagoló.

Rákérdezés-csapda: „Minden komponenst becsomagolj memóba?” — Nem. A shallow-összehasonlításnak ára van, és a legtöbb komponens olcsón renderelődik. Profilozz, aztán a bizonyítottan drágákat memoizáld.

K Hogyan tartasz gyorsan egy nagy bundle-t?

- **Code splitting** `React.lazy` + `Suspense`-szel, vagy route-alapú splitting, hogy a felhasználók csak azt töltsék le, amire egy route-nak szüksége van.
- **Tree shaking** — nevesített tagokat importálj, kerülj a barrel fájlokat, amelyek egész library-eket behúznak, figyelj a mellékhatásos modulokra.
- Elemezz bundle analyzerrel; cseréld le a nehéz függőségeket (pl. `moment` → `date-fns/day.js`).
- Hosszú listáknál **virtualizálj** (`react-window` / `react-virtuoso`), hogy csak a látható sorok renderelődjenek.

K Magyarázd el a „state kiemelése vs colocation” kompromisszumot.

A state-nek a **legalacsonyabb közös ősnél** kell élnie, amelynek szüksége van rá. A túl magasra emelés miatt egy nem kapcsolódó részfa is újrenderelődik minden változásnál (gyakori oka az app-szintű lassúságnak). A state colocation-je oda, ahol használják, csökkenti a render-robbanás sugarát. A senior ösztön: told *lefelé* a state-et, amíg valami fel nem kényszeríti.

4 State-kezelés és adatlekérés

Legyen véleményed, és tudd megvédeni az alternatívákkal szemben.

K Context API vs state library (Redux/Zustand) — hogyan választasz?

A **Context** egy dependency-injection mechanizmus, nem state manager — remek alacsony frekvenciájú globális értékekhez (téma, nyelv, aktuális felhasználó). A gyengesége: minden fogyasztó újrenderelődik, amikor az érték változik, így a nagy frekvenciájú frissítések aknamező.

Nyúlj a **Redux Toolkit**-hez, ha kiszámítható globális state, middleware, time-travel/devtools és csapat-konvenciók kellenek nagy léptékben; a **Zustand/Jotai**-hoz könnyebb, hook-first store-okhoz szelektív feliratkozásokkal. Mondd ki a döntési szabályt: a komplexitás és a frissítési frekvencia dönt, nem a divat.

K Miért választod szét a „server state”-et a „client state”-től?

Mert a szerveradat **aszinkron, megosztott, és elavulhat** — cache-elés, háttérben újralekérés, deduplikáció és invalidáció kell hozzá, amit az általános state managerek nem adnak. Az olyan eszközök, mint a **TanStack Query** vagy az RTK Query, birtokolják ezt az életciklust (loading/error/stale állapotok, cache key-ek, retryk), a client state-et (UI kapcsolók, form state) pedig a `useState` / `context`-re hagyják. A kettő összemosása miatt hackel annyi app buggos fetch-and-cache logikát.

Releváns ehhez a poícióhoz: SharePoint/M365 kontextusban gyakran Graph és SharePoint REST végpontokat fogyasztasz — egy query cache drámaian egyszerűsíti ezt.

K Hogyan kezeled a race condition-öket adatlekérésnél hookokkal?

Ha egy komponens elindít egy kérést, változik a deps, és elindít egy másikat, a válaszok sorrenden kívül érkehetnek meg, és az elavult nyer. Vedd egy **AbortController**-rel vagy egy *ignore flaggel* az effect cleanup-jában:

```
useEffect(() => {
  let ignore = false;
  fetchData(id).then(res => { if (!ignore) setData(res); });
  return () => { ignore = true; }; // cleanup id-váltáskor/unmountkor
}, [id]);
```

Vagy hagyd, hogy egy adatlekérő library kezelje neked a lemondást (cancellation).

5 Modern JavaScript és TypeScript

A mély JS-tudás mindennek az alapja; lehet, hogy közvetlenül az alapokat tesztelik.

K Magyarázd el a closure-öket egy gyakorlati példával.

A closure egy függvény, összecsomagolva azzal a lexikális scope-pal, amelyben definiálták, így megőrzi a hozzáférést azokhoz a változókhoz, miután a külső függvény visszatért. Gyakorlati felhasználások: **adat-priváció** (module pattern, privát state), **függvény-gyáarak** (egy konfigurált logger), **memoizációs cache-ek**, és eseménykezelők/callbackek, amelyek „emlékeznek” az értékekre. Reactben minden render eseménykezelői closure-ök az adott render state-je fölött — pontosan ez a stale-closure bugok forrása.

K Event loop: magyarázd el a microtaskeket vs macrotaskeket.

A JS egyszálú, event loop-pal. A **call stack** futtatja a szinkron kódot; az aszinkron callbackek sorba állnak. A **microtaskek** (Promise `.then`, `queueMicrotask`, `await` folytatások) *teljesen* lecsorognak az aktuális task után és a következő macrotask *előtt*. A **macrotaskek** (`setTimeout`, I/O, UI események) loop-iterációként egyet futnak. Így egy feloldott Promise callback mindig az ugyanakkor sorba állított `setTimeout(0)` előtt fut le.

K == vs === , és hogyan konvertál típust a JS?

A `===` értéket és típust hasonlít, konverzió nélkül. A `==` az absztrakt egyenlőség algoritmust alkalmazza, konvertálva az operandusokat (pl. `'' == 0` az `true`, `null == undefined` az `true`). Szabály: alapból `===`; az egyetlen gyakori védhető `==` az `x == null`, hogy elkapd a `null`-t és az `undefined`-ot is.

K Hogyan kerülöd el a state mutálását, és miért számít ez Reactben?

A React referencia szerint érzékeli a változást (`Object.is`). Egy objektum/tömb helyben mutálása megtartja ugyanazt a referenciát, így a React (és a `memo`) „nincs változás”-t lát, és kihagyja az update-et — vagy rosszabb, kiszámíthatatlanul frissít. Hozz létre **új referenciákat**: `spread` (`{...obj, x}`, `[...arr, item]`), `map` / `filter`, vagy egy library, mint az **Immer** a mély update-ekhez. Ezért is használ a Redux Toolkit `Immer`-t a motorháztető alatt.

K TypeScript: generikusok, és mikor használod őket?

A generikusok lehetővé teszik újrahasználatát, típusbiztos kód írását egy típussal parametrizálva — a típus átfolyik, helyett, hogy fixen vagy `any`-ként lenne rögzítve. Klasszikus példa: egy `useFetch<T>()` hook, aminek a visszatérése a hívó alakjára tipizált, vagy egy `List<T>` komponens. Említs gyakorlati eszközöket: **union és discriminated-union típusok** state machine-ekhez, **utility típusok** (`Partial`, `Pick`, `Omit`, `Record`), és az `unknown` preferálása az `any` helyett a határokon.

Rákérdezés-csapda: „any vs unknown?” — az `any` kikapcsolja az ellenőrzést és csendben terjed; az `unknown` típusbiztos — használat előtt szűkítened kell. Használj `unknown`-t nem megbízható inputhoz (API-válaszok).

K Magyarázd el az `async/await`-et vs `Promise`-ok, és a hibakezelést.

Az `async/await` szintaktikai cukor a `Promise`-ok felett, ami szekvenciálisan olvasható. Hibák: csomagold az `await`-et `try/catch`-be; párhuzamossághoz használj `Promise.all`-t (gyorsan bukik) vagy `Promise.allSettled`-t (összegyűjti az összes eredményt/hibát). Gyakori senior pont: ne `await`-elj egy ciklusban, ha a hívások függetlenek — indítsd őket együtt, és `await Promise.all`-al várj rájuk, hogy elkerüld a latencia sorbarendezeit.

6 Tesztelés — Jest és React Testing Library

A hirdetés „essential”-nek nevezi. Számíts konkrét, gyakorlati kérdésekre itt.

K Mi a React Testing Library alapfilozófiája?

Viselkedést tesztelj, ne implementációt. Úgy queryzz és assertálj, ahogy egy felhasználó megtapasztalja a UI-t — role, label és szöveg alapján — nem a komponens belsőiségei, state-je vagy CSS-osztályai szerint. A haszon: a tesztek túlélnek a refaktorokat (átírhatod a belsőt, és a tesztek átmennek, amíg a viselkedés áll). A vezérelv: „minél jobban hasonlítanak a tesztjeid arra, ahogy a szoftvert használják, annál több bizalmat adnak.”

K Milyen queryket használsz, és milyen prioritásban?

Az RTL által ajánlott prioritási sorrend:

- **`getByRole`** (`name`-mel) — a legközelebb ahhoz, ahogy a felhasználók és a segítő technológiák navigálnak; az alapértelmezett választás.
- **`getByLabelText`** — űrlapmezők.
- **`getByPlaceholderText`, `getByText`** — nem interaktív tartalomhoz.
- **`getByTestId`** — végső megoldás, ha semmi szemantikus nem illik.

És a változatok: a `getBy*` dob, ha nincs meg, a `queryBy*` null-t ad vissza (hiány assertálásához), a `findBy*` `Promise`-t ad vissza (aszinkron megjelenéshez).

K Hogyan tesztelsz valami aszinkront — adat, ami egy `fetch` után jelenik meg?

Használj `findBy*`-ot vagy `waitFor`-t, amelyek újrapróbálkoznak, amíg az assertion átmegy vagy lejár az időkorlát:

```
test('betöltés után megjeleníti a usereket', async () => {
  render(<UserList />);
  // a findBy megvárja, míg az elem megjelenik
  expect(await screen.findByText('Ada Lovelace')).toBeInTheDocument();
});
```

Soha ne használj önkényes `setTimeout`-ot a tesztekben. A felhasználói interakciókat csomagold `userEvent`-be (ami aszinkron), és `await`-eld őket.

K `userEvent` vs `fireEvent` — melyik és miért?

Preferáld a `@testing-library/user-event`-et. A `fireEvent` egyetlen nyers DOM-eseményt küld; a `userEvent` a teljes interakciót szimulálja (egy kattintás pointerdown/focus/mouseup-ot is kivált, a gépelés karakterenként keydown/keypress/input-ot). Sokkal közelebb van a valós felhasználói viselkedéshez, és elkapja azokat a bugokat, amiket a `fireEvent` nem.

K Hogyan mockolsz modulokat, timereket és hálózati hívásokat Jestben?

- **Modulok:** `jest.mock('./api')`, vagy `jest.spyOn(obj, 'method')` egyetlen függvény megfigyeléséhez/stubolásához.
- **Timerek:** `jest.useFakeTimers()` + `jest.advanceTimersByTime()` a debounce-ok/intervallumok determinisztikus teszteléséhez.
- **Hálózat:** a best practice az **MSW (Mock Service Worker)** — a hálózati rétegen fog el, így a komponensek valódi fetch-logikát futtatnak mockolt válaszok ellen, ahelyett, hogy magát a `fetch`-et mockolnád.

Senior jelzés: Monddd, hogy kerülöd a túl-mockolást. Ha mindent mockolsz, a mockjaidat teszteled, nem az appodat. A határt mockold (hálózat, idő), a többi tartd valódinak.

K Unit vs integrációs vs snapshot tesztek — hogyan egyensúlyozol köztük?

Preferáld az **integrációs stílusú** teszteket, amelyek egy komponenst a gyerekeivel együtt renderelnek, és felhasználó által látható viselkedést assertálnak — a legjobb bizalom tesztenként. Tiszta **unit** tesztek komplex logikához/utility-khez/saját hookokhoz (`renderHook`). **Snapshot** tesztek takarékosan — a nagy snapshotok rothadnak, vakon frissülnek, és semmi értelmeset nem assertálnak; kis, szándékos snapshotok csak stabil kimenethez. Kritikus utak értelmes lefedettségét hajszold, ne egy 100%-os számot.

7 Skálázható architektúra és komponenskönyvtárak

A pozíció kifejezetten említi a megosztott komponenskönyvtárakat és a skálázható architektúrát — legyen kész véleményed.

K Hogyan strukturálsz egy nagy React kódbázist?

Válts a típus-alapú mappákról (`/components`, `/hooks`) a **feature-alapú / domain mappákra**, amelyek egy feature komponenseit, hookjait, tesztjeit és state-jét colocatálják. Tarts egy megosztott `/ui` vagy design-system réteget a primitíveknek. Kényszerítsd ki a **függőségi irányt**: a feature-ök a megosztottól függenek, sosem fordítva; kerüld a feature-ök közötti importokat. Említsd a barrel-fájl fegyelmet és a világos publikus API-kat modulonként, hogy a refaktorok kontroll alatt maradjanak.

K Hogyan tervezel újrahasználató komponenst egy megosztott könyvtárhoz?

- **Kompozíció konfiguráció helyett** — al-komponenseket és `children`-t exponálj egy óriási prop-objektum helyett (compound komponensek, pl. `<Select><Select.Option/></Select>`).
- Értelmes alapértelmezések, de engedj felülírást — `className` átadás, `as` /polimorf propok, továbbított refek.
- **Controlled + uncontrolled** támogatás, hogy a fogyasztók választhassanak.
- Beépített akadálymentesség (role-ok, billentyűzet, fókuszkezelés), erős TypeScript típusok, és semmi rejtett kapcsolódás app-specifikus state-hez.

K Mi az Error Boundary, és hova helyezed?

Egy osztály-komponens (vagy egy library, mint a `react-error-boundary`), ami `getDerivedStateFromError` / `componentDidCatch`-et implementál, és **elkapja a render-fázis hibáit a részfájában**, és fallbacket mutat, ahelyett, hogy az egész appot unmountolná. Helyezd őket értelmes határookra — route-onként, nagyobb widgetenként — hogy egy hibázó panel ne fehéritse ki az egész oldalt. Említsd a korlátokat: nem kapják el az eseménykezelők, aszinkron kód vagy SSR hibáit.

K Hogyan verziózol és vezetsz be változásokat egy megosztott komponenskönyvtárba?

Szemantikus verziózás egy fegyelmezett breaking-change definícióval (prop eltávolítás/átnevezés, viselkedésváltozás). Szállíts changelogot, deprecálj eltávolítás előtt, és adj codemodokat nagy migrációkhoz. Dokumentáld a komponenseket **Storybookban** vizuális/interakciós tesztekkel a regressziók elkapásához. Monorepóban az olyan eszközök, mint a Changesets, kezelik a verziózást; a fogyasztók szándékosan frissítenek, nem erővel törnek el.

8 SPFx és SharePoint Online

A megkülönböztető tényező ennél a konkrét pozíciónál. Már a „némi tapasztalat” is kitűnik, ha magabiztosan válaszolsz — de légy pontos abban, amit tudsz.

K Mi a SharePoint Framework (SPFx), és miért használnád?

Az SPFx a Microsoft **kliens-oldali fejlesztési modellje** a SharePoint Online és a Microsoft 365 kiterjesztéséhez. A kódod **az oldal és az aktuális felhasználó kontextusában** fut (nincs iframe, böngésző-oldalon), így a komponensek gyorsan töltődnek és automatikusan tiszteletben tartják a felhasználó jogosultságait. Toolchain-alapú (Node, Yeoman generátor, gulp/webpack), és a React out-of-the-box támogatott — a Microsoft saját modern SharePoint UI-ja Reacttel és Fluent UI-jal épült.

K Mi a különbség egy Web Part és egy Extension között?

- **Client-Side Web Partok** — a fő építőelem: újrahaználható komponensek, amiket a felhasználók hozzáadnak és konfigurálnak egy oldalon (property pane-nel a beállításokhoz).
- **Extension-ök** — magát a SharePoint élményt szabják testre: *Application Customizerek* (header/footer injektálása minden oldalra), *Field Customizerek* (egy lista-oszlop renderelése), *Command Setek* (lista-toolbar akciók hozzáadása).

K Vezess végig egy SPFx web part lifecycle-jén.

Az `onInit()` futtatja a setupot (context lekérése, auth, service kliensek) az első render előtt, és Promise-t ad vissza. A `render()` a web part DOM-elemébe rajzol — egy React web partnál itt `ReactDOM.render`-eled a gyöker-komponensedet. A **property pane** metódusok (`getPropertyPaneConfiguration`, `onPropertyPaneFieldChanged`) kezelik a konfigurálható beállításokat. Az `onDispose()` takarít. Lényeg: egy standalone SPA-val ellentétben a web part egy komponens, amit a SharePoint oldal-lifecycle-je hostol.

K Hogyan kérsz le SharePoint / Microsoft Graph adatot SPFx-ből?

Használd a context által biztosított HTTP klienseket: **SPHttpClient** a SharePoint REST-hez, **MSGraphClientV3** a Graph-hoz — mindkettő kezeli neked az auth tokeneket. A gyakorlatban a legtöbb csapat a **PnPjs**-t használja, egy fluent wrappert, ami sokkal tisztábbá teszi a lista/item/Graph műveleteket (pl. `sp.web.lists.getByTitle('X').items()`). A Graph/API-khoz a jogosultságokat a tenant **API access** oldalán adják meg (Azure AD app permissionök).

Ha mélységre kérdeznék: Légy őszinte a szintedről („dolgoztam SPFx web partokkal és PnPjs-sel lista-adatokhoz; gyorsan felveszem azokat a területeket, amiket még nem érintettem”). A magabiztos + pontos jobb, mint a túligérés — a SharePoint-fejlesztők azonnal kiszúrják a blöffölést.

K Hogyan kezeli az SPFx a jogosultságokat és a biztonságot egy standalone SPA-hoz képest?

Az SPFx a **bejelentkezett felhasználóként** fut, így örökli annak SharePoint-jogosultságait — nem kell külön auth flow-t kezelned a SharePoint-adatokhoz. Külső/Graph API-khoz olyan scope-okat kérsz, amiket egy tenant admin központilag jóváhagy. A rád maradó biztonsági felelősségek: ne exponálj secreteket kliens-kódban, sanitizáld az injektált HTML-t, tiszteld a least-privilege-et a kért

scope-oknál, és validáld, hogy a UI szépen degradálódik, ha egy felhasználónak nincs hozzáférése egy elemhez.

K Hogyan bundle-özöl, deployolsz és frissítesz egy SPFx megoldást?

A build egy **.sppkg csomagot** állít elő, amit a tenant **App Catalogjába** töltesz fel; az assetek hostoltak (Office 365 CDN / SharePoint library). Az adminok deployolják/jóváhagyják, aztán elérhető lesz oldalakhoz hozzáadni. A frissítések egy új csomag-verziót szállítanak. Említsd, hogy gondosan verziózol, és először egy dev/test site collectionben tesztelsz — illeszkedve a „development, test, and acceptance environments”-hez, amit a hirdetés leír.

9 Akadálymentesség és frontend minőség

A hirdetés explicit említi az accessibility elveket — egy gyors, hiteles válasz lefedi ezt.

K Hogyan építesz akadálymentes React komponenseket?

- Használj **szemantikus HTML-t először** (`<button>` , `<nav>` , `<label>`) — ez ingyen ad role-okat és billentyűzet-viselkedést; az ARIA-hoz csak a hézagok kitöltéséhez nyúlj.
- **Billentyűzet-kezelhetőség** — minden interaktív elem elérhető és használható billentyűzettel, látható fókusz-állapotok, logikus tab-sorrend, fókuszkezelés modálokban.
- Címkezd az űrlapmezőket, adj `alt` szöveget, tartsd a szín-kontrasztot, és jelezd a dinamikus változásokat `aria-live` régiókkal.
- Tesztelj **jest-axe**-szel unit tesztekben és egy képernyőolvasóval / Lighthouse-szal valódi ellenőrzésekhez.

K Milyen frontend biztonsági problémákat tartasz szem előtt?

Az **XSS** a legnagyobb — a React alapból escape-eli a szöveget, de a `dangerouslySetInnerHTML` újrainyítja az ajtót, így sanitizáld (pl. DOMPurify) az injektálandó HTML-t. Továbbá: soha ne bízz csak a kliens-oldali validációban, tartsd a secretet/tokeneket a bundle-ön kívül, légy óvatos a `target="_blank"` -kel (`rel="noopener"`), és tiszteld az app Content Security Policy-jét. SharePointban konkrétan tiszteld a tenant-biztonságot és a least-privilege API scope-okat.

K Hogyan magyaráznál el egy technikai kompromisszumot egy nem-technikai stakeholdernek? (rá fognak kérdezni — benne van a profilban)

Fogalmazd meg **költség, kockázat és eredmény** mentén, ne szakzsargonban. Példa: „Az A opció egy hét alatt kész, de később nehezebb lesz változtatni; a B opció három hetet vesz igénybe, de támogatja azokat a funkciókat, amikről tudjuk, hogy jönnek a következő negyedévben.” Adj ajánlást indoklással, kínáld fel a döntést, és erősítsd meg a megértést. Az interjúztatók azt mérik, megbízhatnak-e benned business analystek és ügyfelek előtt — a világosság és a keretezés annyit számít, mint a technikai tartalom.